Università degli Studi di Napoli Federico II

Esame di Programmazione II

Prova pratica del giorno 01/07/2014
Durata della prova: 120 minuti

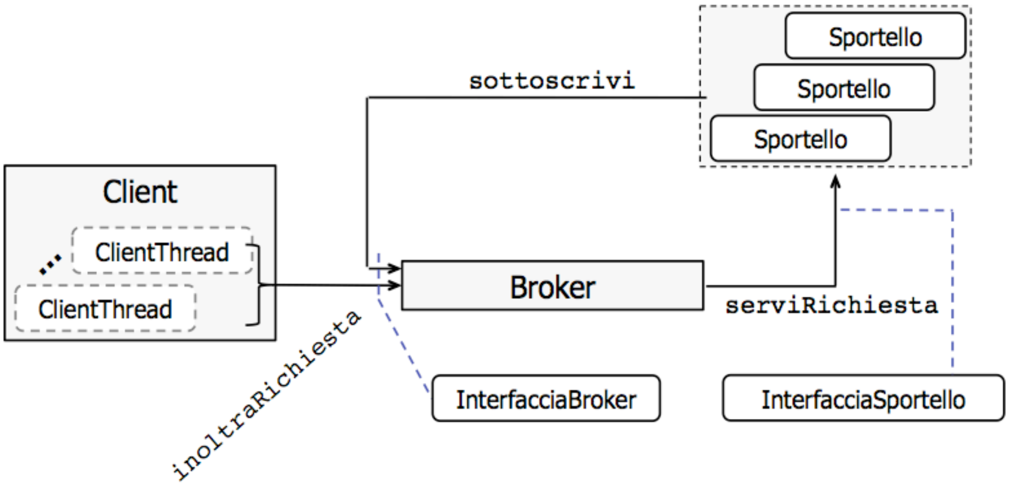
Cognome……………...…….………. Nome…………….………………. Matr……………

°°

Lo studente legga attentamente il testo e produca il programma ed i casi di test necessari per dimostrarne il funzionamento. La mancata compilazione completa dell'elaborato, la compilazione con errori o l'esecuzione errata daranno luogo alla valutazione come ***prova non superata****.*
Ricordarsi di indicare Cognome, Nome e matricola su questo stesso foglio, che dovrà essere in ogni caso consegnato alla Commissione. Al termine della prova lo studente dovrà far verificare il funzionamento del programma ad un membro della Commissione.

°°

Testo della prova



Il candidato realizzi un broker per la gestione degli sportelli di una filiale bancaria. Il sistema si compone di **tre entità**, come mostrato in figura:

1. **Sportello**: offre il servizio `boolean` **`serviRichiesta`**`(int id_cliente)`. Al proprio avvio, lo Sportello si sottoscrive al Broker utilizzando il servizio **`sottoscrivi`**.
   Uno Sportello è invocato dal Broker tramite il metodo **`serviRichiesta`**. Ogni invocazione di **`serviRichiesta`** dura un tempo scelto a caso tra 5 e 10 secondi: **`serviRichiesta`** stampa a video `id_cliente` e restituisce esito **`true`**.

   Un solo thread per volta può eseguire la **`serviRichiesta`**. Se lo Sportello sta già eseguendo una richiesta, il thread che ha invocato **`serviRichiesta`** viene messo in attesa. Al più 2 thread possono essere accodati nell'attesa che lo Sportello si liberi: ulteriori richieste terminano immediatamente con esito **`false`.**

2. **Broker:** riceve le richieste del Client tramite `boolean` **`inoltraRichiesta`**`(int id_cliente).` Il Broker ha una lista di Sportelli ordinati nell'ordine di sottoscrizione. Il Broker invoca in ordine tutti gli sportelli finché ne viene individuato uno che non restituisce **`false`**. Se tutti gli Sportelli restituiscono **`false`**, **`inoltraRichiesta`** restituirà **`false`**.

3. **Client**: genera T thread, ognuno dei quali, allo scadere di un tempo di t secondi (con t scelto a caso tra 1 e 3), invoca **`inoltraRichiesta`**. Ogni thread genera R richieste. Si imposti T pari a 10; R pari a 5. Si selezioni `id_cliente` in maniera casuale tra 1 e 100.

Il candidato utilizzi le socket ed il pattern proxy/skeleton (obbligatorio).